Event-Based Communication (StateFlow) and Core Implementation

The application uses StateFlow as the reactive backbone for all data updates, decoupling business logic from UI refreshing.

Core Implementation:

TicketRepository acts as the Single Source of Truth (SSOT):

private val _tickets = MutableStateFlow(MockTickets.tickets) val tickets: StateFlow<List<Ticket>> = _tickets.asStateFlow()

TicketListViewModel transforms, filters, and sorts the stream dynamically:

val tickets: StateFlow<List<Ticket>> = ticketRepository.tickets .map { list -> list.sortedByDescending { it.priority.level } } .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), emptyList())

Scenario 1 — Ticket Creation Flow

1. The user inputs incident data and presses "Crear ticket".
2. CreateTicketViewModel.createTicket() invokes ticketRepository.createTicket(ticket).
3. The repository updates the collection: _tickets.value = listOf(newTicket) + currentList.
4. All active subscribers receive the emission automatically.
5. TicketListScreen triggers recomposition; the newly added ticket instantly appears at the top.

Scenario 2 — Priority Shifting & Auto-Sorting

1. The user alters a ticket's priority level inside TicketDetailScreen.
2. TicketDetailViewModel.updatePriority() calls ticketRepository.updatePriority(id, priority).
3. The repository updates the list and emits it.
4. TicketListScreen intercepts the updated flow, reapplies the sorting logic (sortedByDescending), and the item moves to its brand-new position in real-time.